



BSCYS-F24-B

PROJECT

Secure Software Design and Development

1st June, 2026

BSCYS-F-24B | Group 3

HAANI BIN QAISER

M. ABAID

M. ASIM SALEEM

M. DAWOOD NAZEER

M. RIZWAN

PROJECT – AUTOMATED CRYPTOGRAPHIC PIPELINE VIA JENKINS & DOCKER

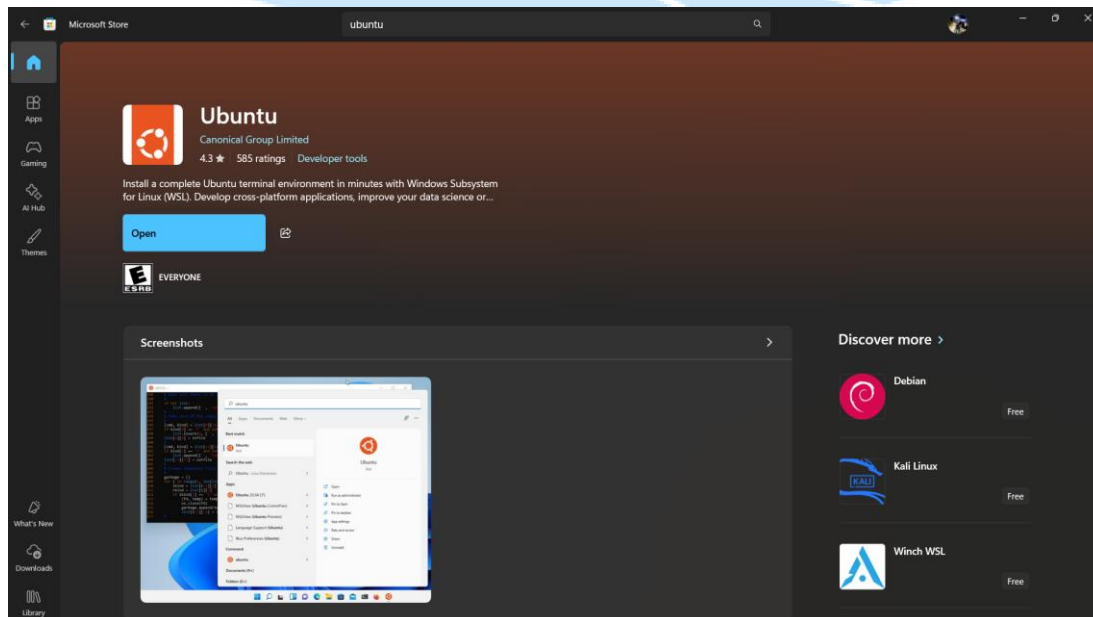
Requirements:

- Docker on WSL
- Docker Desktop
- Jenkins on Windows
- Git on Windows
- Github Repository

Requirements Setup:

Docker on WSL:

First make sure that you have installed Docker on WSL ubuntu. For that first install Ubuntu from Microsoft store, it will install WSL by itself.



Then install docker on WSL by running the command “`sudo apt install docker`”. Then check its version by using the command “`docker version`” or “`docker -version`”.

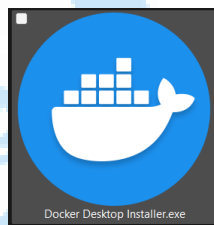
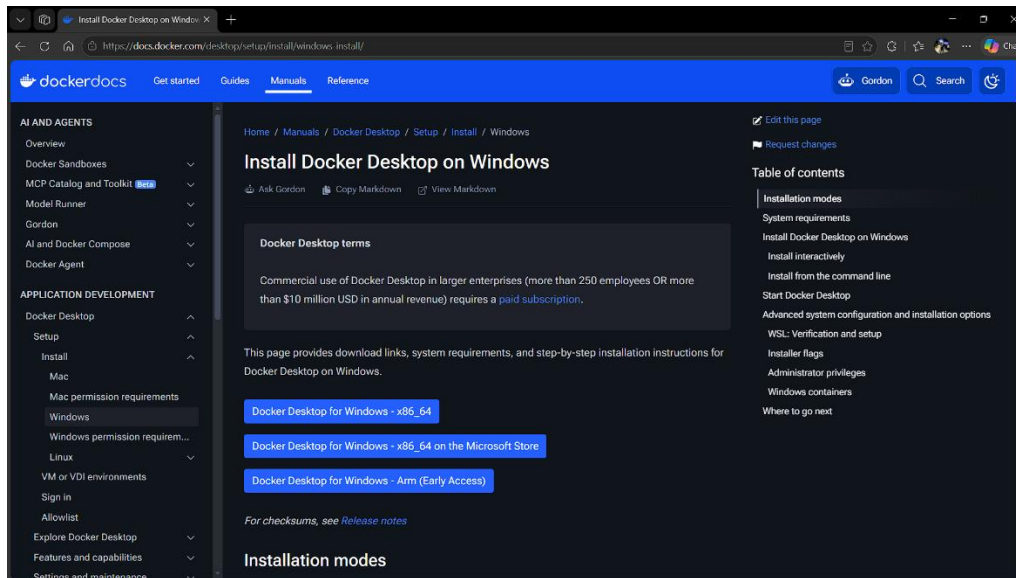
```
arison411@DESKTOP-1SL1357: x + v
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\HP>docker version
'docker' is not recognized as an internal or external command,
operable program or batch file.

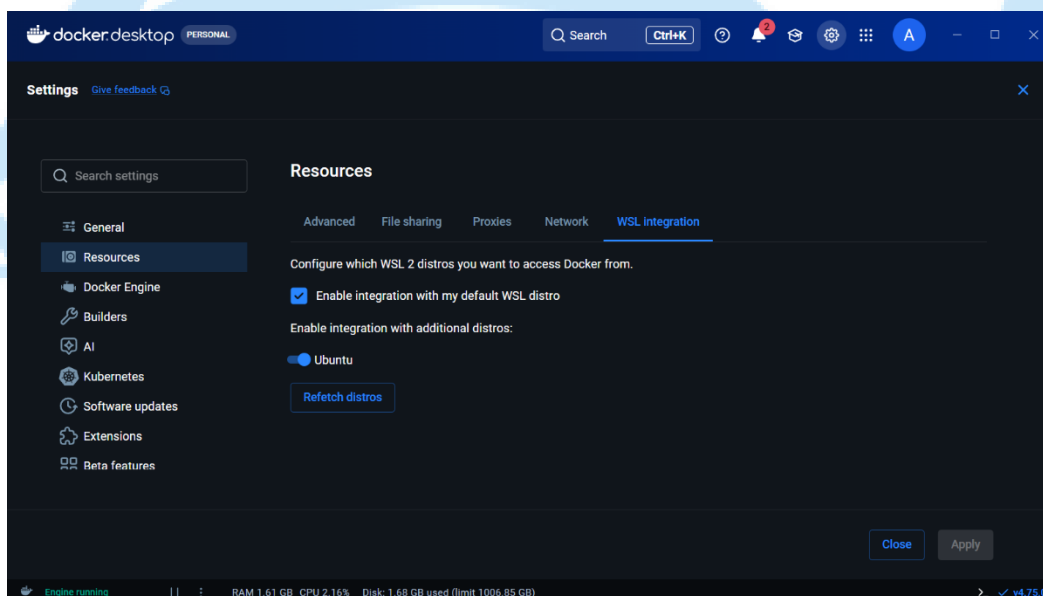
C:\Users\HP>wsl
arison411@DESKTOP-1SL1357: /mnt/c/Users/HP$ docker version
Client:
Version:           29.1.3
API version:       1.52
Go version:        go1.24.4
Git commit:        29.1.3~0ubuntu3~24.04.2
Built:            Wed Apr 29 16:41:06 2026
OS/Arch:          linux/amd64
Context:          default
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
arison411@DESKTOP-1SL1357: /mnt/c/Users/HP$ |
```

Docker Desktop:

You can download docker from its website. Install it with the default options and select WSL as the base for Docker.

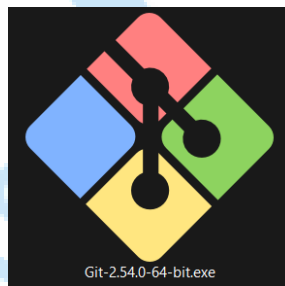
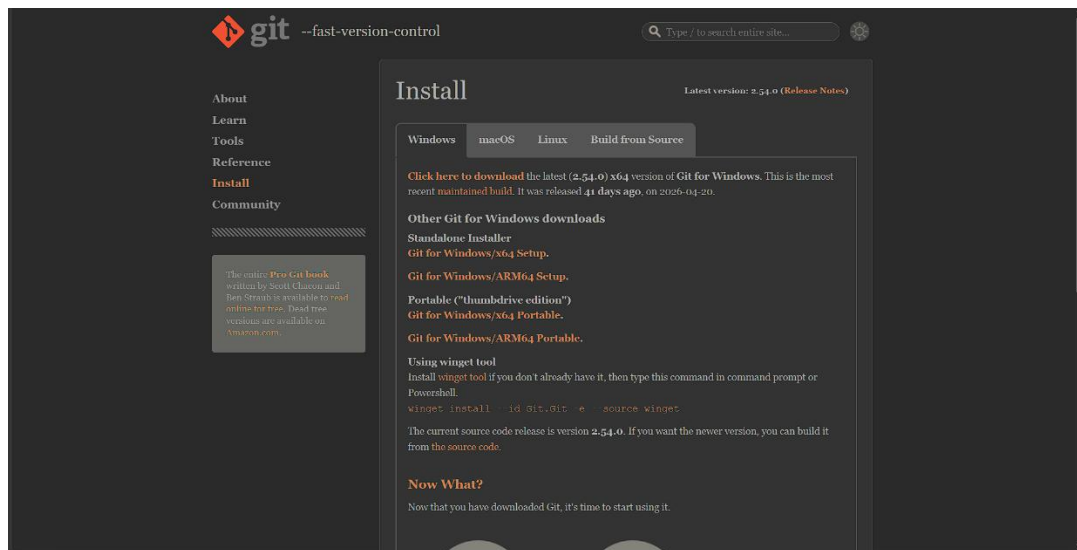


Then go to settings -> resources and then Enable integration with my default WSL distro and enable your machine (Ubuntu).



Git:

Download Git setup from its website.



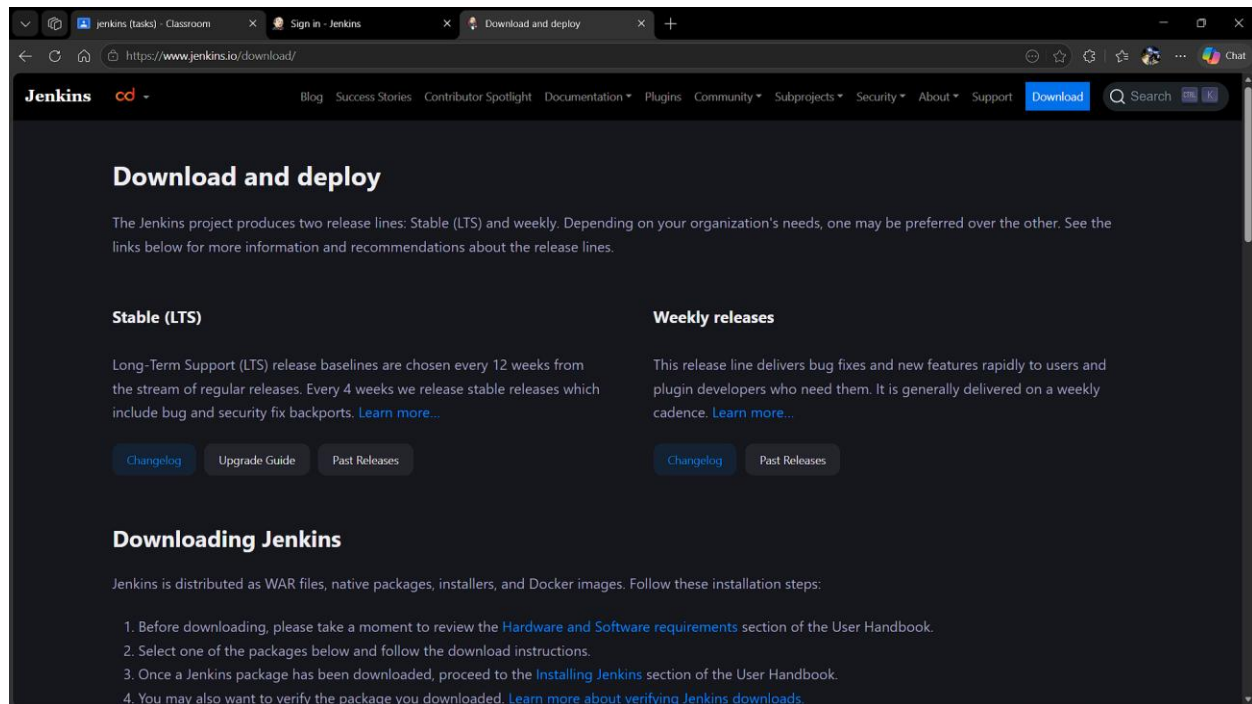
Install it with its default options and complete the setup. Then Sign in with your Account.

Jenkins:

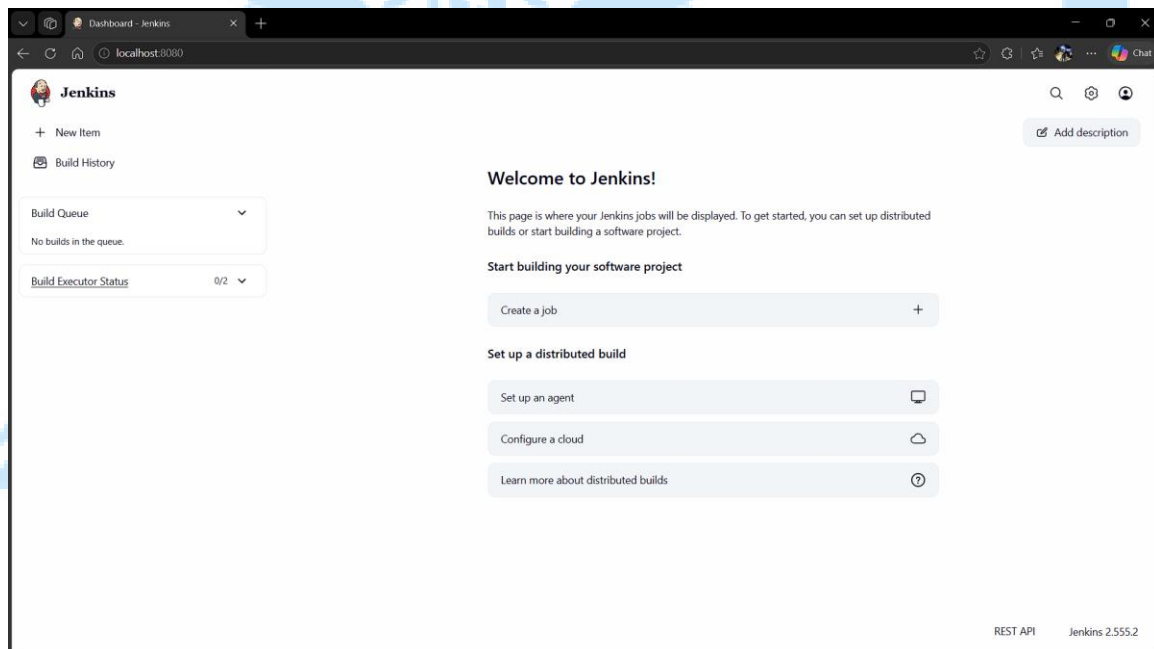
First Install "Java Development kit" on the system.



Then download and install Jenkins from its website.



After its installation is complete, create its account and install its required services.
After that it is ready to use.



After that go to settings->tools and scroll down to Git installations, make sure that git installation is added in the tools.

Git

Name

Default

Path to Git executable ?

git.exe

☐ Install automatically ?

+ Add Git

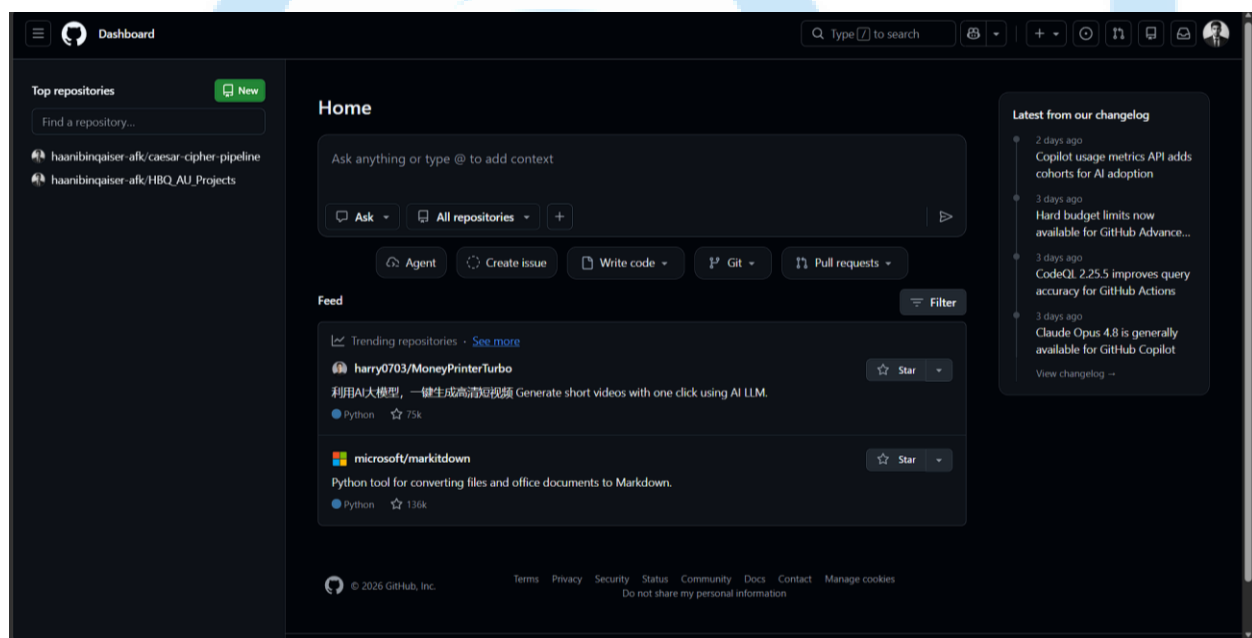
Then go to settings->plugins and install the given plugins.

[Report an issue with this plugin](#)

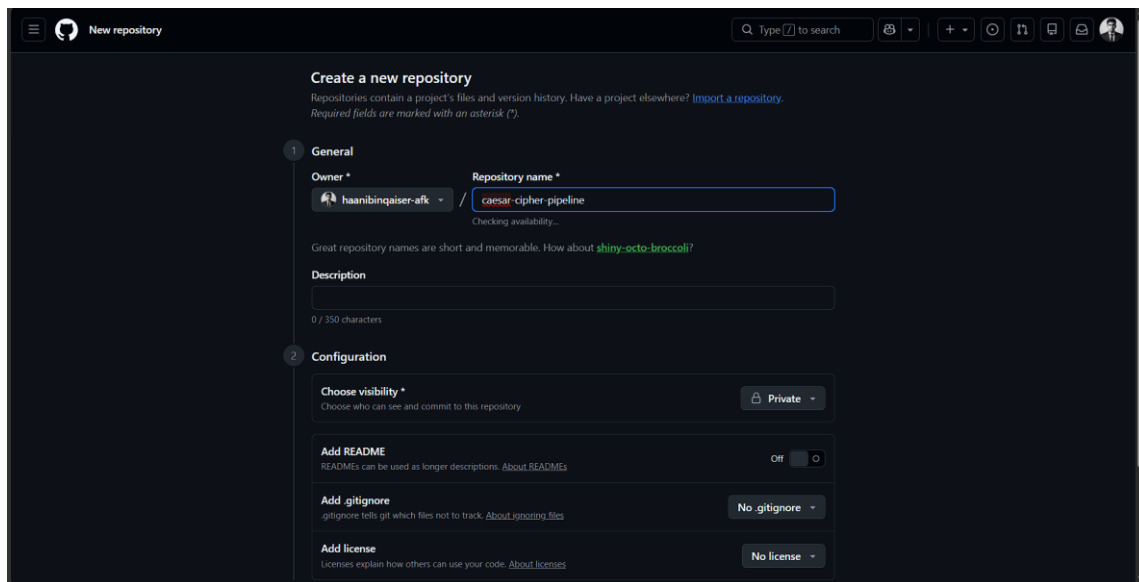
[Report an issue with this plugin](#)

[Report an issue with this plugin](#)

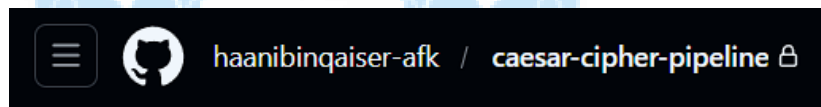
Now go to Github website. Login and access your profile.



Then create a new repository and keep it private. Keep all the option off. For my case, I have kept its name “caesar-cipher-pipeline”. It should be simple so that Jenkins can easily access it.

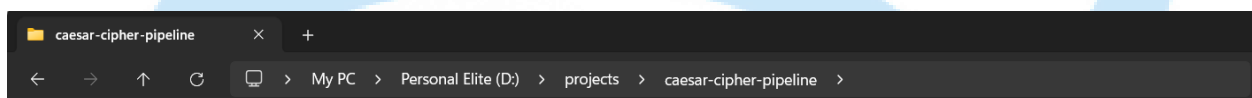


And finally, the repository is also created successfully.



Project Implementation:

First select a directory for your project on Windows. For my project, I have selected the directory “D:/projects/caesar-cipher-pipeline”. Keep the directory name same as the repository name to avoid confusion.



Since Docker Desktop is installed, Docker might be easily accessible from windows. To verify it, open the terminal and run the command “docker --version”.

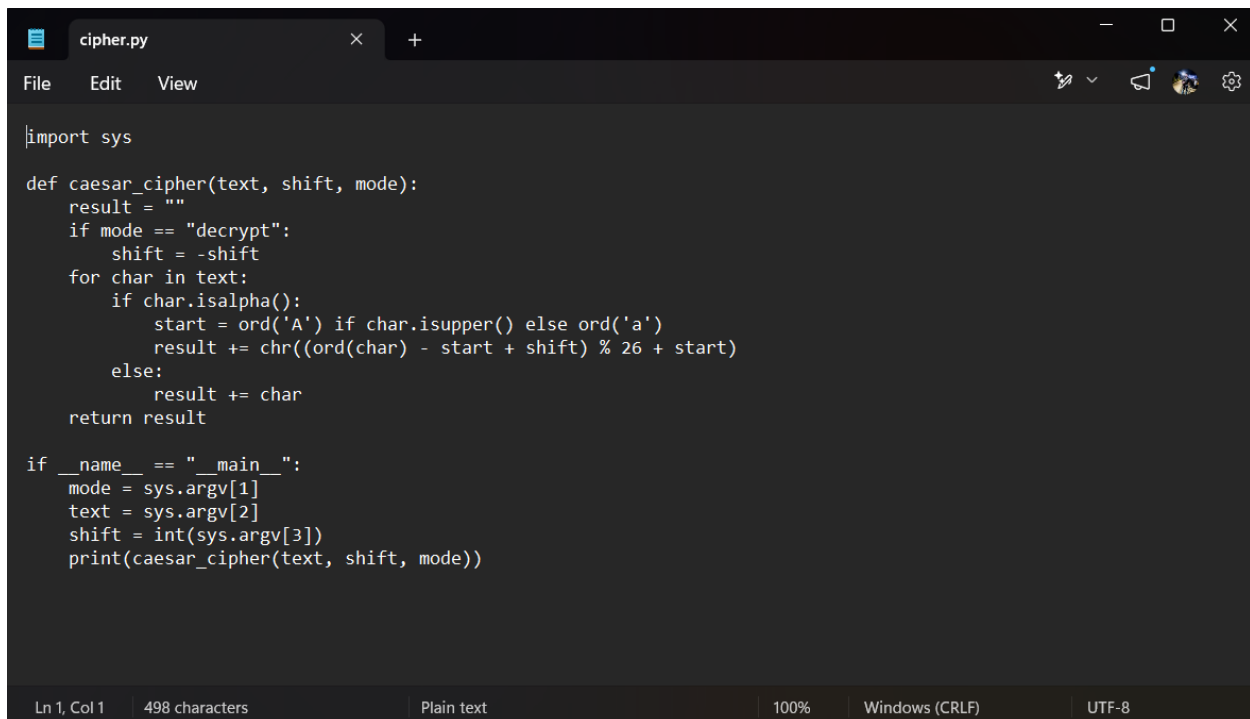
```
PS D:\projects\caesar-cipher-pipeline> docker --version
Docker version 29.5.2, build 79eb04c
```

Now in that directory, create 3 files: Dockerfile, Jenkinsfile, and cipher.py. Make sure that the alphabets are same as they are here and the extensions are also correct. Dockerfile and Jenkinsfile will not have any extensions.

 cipher.py	5/31/2026 8:15 PM	Python File	1 KB
 Dockerfile	5/31/2026 8:17 PM	File	1 KB
 Jenkinsfile	5/31/2026 8:17 PM	File	2 KB

Then edit these files:

Cipher.py:

A screenshot of a code editor window titled 'cipher.py'. The editor has a dark theme and shows Python code for a Caesar cipher. The code includes a function 'caesar_cipher' that takes 'text', 'shift', and 'mode' as arguments. It handles both encryption and decryption by shifting the ASCII values of alphabetic characters. A main block at the bottom uses 'sys.argv' to parse command-line arguments for mode, text, and shift, then prints the result of the cipher function. The status bar at the bottom indicates 'Ln 1, Col 1', '498 characters', 'Plain text', '100%', 'Windows (CRLF)', and 'UTF-8'.

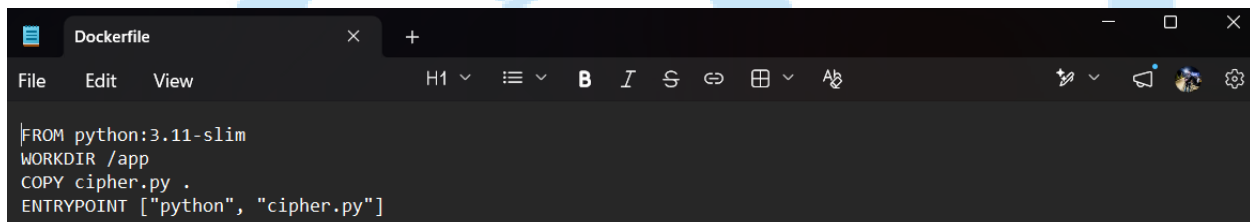
```
import sys

def caesar_cipher(text, shift, mode):
    result = ""
    if mode == "decrypt":
        shift = -shift
    for char in text:
        if char.isalpha():
            start = ord('A') if char.isupper() else ord('a')
            result += chr((ord(char) - start + shift) % 26 + start)
        else:
            result += char
    return result

if __name__ == "__main__":
    mode = sys.argv[1]
    text = sys.argv[2]
    shift = int(sys.argv[3])
    print(caesar_cipher(text, shift, mode))
```

This file should contain the code to implement Caesar Cipher Algorithm.

Dockerfile:

A screenshot of a code editor window titled 'Dockerfile'. The editor has a dark theme and shows a Dockerfile with four instructions: 'FROM python:3.11-slim', 'WORKDIR /app', 'COPY cipher.py .', and 'ENTRYPOINT ["python", "cipher.py"]'. The status bar at the bottom shows 'H1', a list icon, 'B', 'I', 'S', '↺', a grid icon, and 'Ab'.

```
FROM python:3.11-slim
WORKDIR /app
COPY cipher.py .
ENTRYPOINT ["python", "cipher.py"]
```

Dockerfile should contain the list of required elements for Docker to run the task.

Jenkinsfile:

```
pipeline {
    agent any

    stages {
        stage('Verify Docker Accessibility') {
            steps {
                echo "Current Branch: ${env.BRANCH_NAME}"
                bat 'docker --version'
            }
        }

        stage('Build Cipher Image') {
            steps {
                echo "Building the Caesar Cipher Docker executor..."
                bat "docker build -t caesar-cipher:latest ."
            }
        }

        stage('Encryption Stage') {
            steps {
                echo "Original Text: DevOps with Jenkins, Docker Desktop, and WSL!"
                echo "Encrypting..."
                script {
                    def output = bat(
                        script: "docker run --rm caesar-cipher:latest encrypt \"DevOps with Jenkins, Docker Desktop, and WSL!\" 7",
                        returnStdout: true
                    ).trim()
                    def lines = output.split(/\r?\n/)
                    env.ENCRIPTED_TEXT = lines[-1]
                }
                echo "Encrypted Output: ${env.ENCRIPTED_TEXT}"
            }
        }

        stage('Decryption Stage') {
            steps {
                echo "Passing Encrypted Text to Decryption Stage..."
                script {
                    def output = bat(
                        script: "docker run --rm caesar-cipher:latest decrypt \"${env.ENCRIPTED_TEXT}\" 7",
                        returnStdout: true
                    ).trim()
                    def lines = output.split(/\r?\n/)
                    def decrypted_text = lines[-1]
                    echo "Decrypted Output: ${decrypted_text}"
                }
            }
        }
    }

    post {
        always {
            echo "Cleaning up local Docker images..."
            bat "docker rmi caesar-cipher:latest --force"
        }
    }
}
```

Jenkinsfile should contain the code to automate the whole task and also to display the output on Jenkins.

After the files are created and their content is also added. Then initialization this project directory as a git repository. But for that First Setup your id and email to enable proper git usage.

```
PS D:\projects\caesar-cipher-pipeline> git config --global user.email "haanibinqaiser@gmail.com"
PS D:\projects\caesar-cipher-pipeline> git config --global user.name "Haani Bin Qaiser"
```

Then you can easily initialize the git repository.

```
PS D:\projects\caesar-cipher-pipeline> git init
Initialized empty Git repository in D:/projects/caesar-cipher-pipeline/.git/
PS D:\projects\caesar-cipher-pipeline> git add .
PS D:\projects\caesar-cipher-pipeline> git commit -m "Set up Caesar Cipher with Jenkinsfile and Dockerfile on D drive"
[master (root-commit) 648a954] Set up Caesar Cipher with Jenkinsfile and Dockerfile on D drive
3 files changed, 80 insertions(+)
create mode 100644 DockerFile
create mode 100644 JenkinsFile
create mode 100644 cipher.py
PS D:\projects\caesar-cipher-pipeline> git add Jenkinsfile cipher.py Dockerfile
```

Make sure the case sensitivity of files is kept same as here. Otherwise it may cause errors and will need to be fixed.

```

PS D:\projects\caesar-cipher-pipeline> git mv JenkinsFile Jenkinsfile_temp
PS D:\projects\caesar-cipher-pipeline> git mv Jenkinsfile_temp Jenkinsfile
PS D:\projects\caesar-cipher-pipeline> git commit -m "Force Git to update case-sensitivity of Jenkinsfile"
[master 6ad81ee] Force Git to update case-sensitivity of Jenkinsfile
1 file changed, 0 insertions(+), 0 deletions(-)
rename JenkinsFile => Jenkinsfile (100%)
PS D:\projects\caesar-cipher-pipeline> git ls-tree -r HEAD --name-only
DockerFile
Jenkinsfile
cipher.py
PS D:\projects\caesar-cipher-pipeline> git mv DockerFile Dockerfile_temp
PS D:\projects\caesar-cipher-pipeline> git mv Dockerfile_temp Dockerfile
PS D:\projects\caesar-cipher-pipeline> git commit -m "Fix case-sensitivity typo in Dockerfile name"
[main 7e6260c] Fix case-sensitivity typo in Dockerfile name
1 file changed, 0 insertions(+), 0 deletions(-)
rename DockerFile => Dockerfile (100%)

```

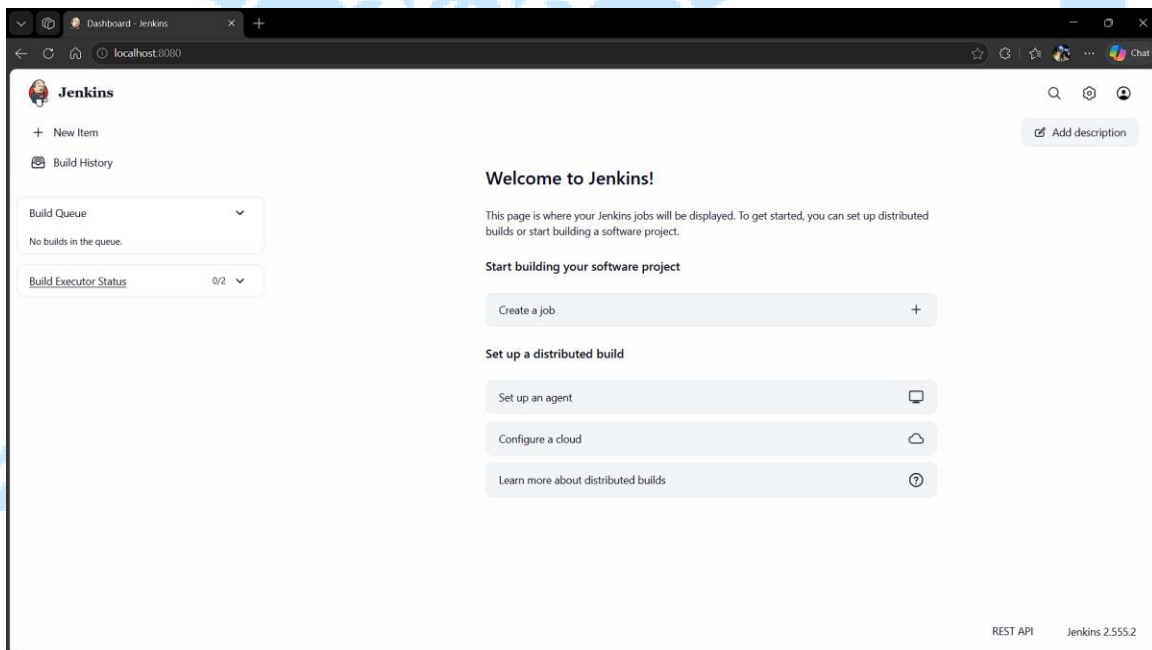
Now this git repository can also be used in the Jenkins project, but it may cause errors due to Windows security limitations. To avoid all of these issues, it is better to push this repository to Github (to the repository we created recently).

```

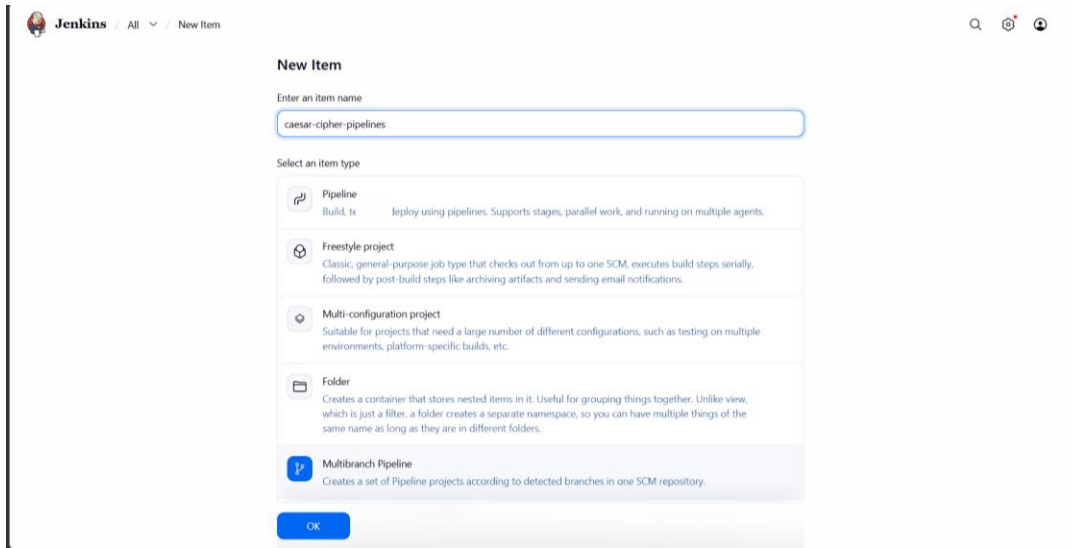
PS D:\projects\caesar-cipher-pipeline> git remote add origin https://github.com/haanibinqaiser-afk/caesar-cipher-pipeline.git
PS D:\projects\caesar-cipher-pipeline> git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 1.37 KiB | 467.00 KiB/s, done.
Total 7 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/haanibinqaiser-afk/caesar-cipher-pipeline.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

```

Now open Jenkins and sign in.



Then Create a job. Here, name the job “caesar-cipher-pipeline” and select the multibranch pipeline type, then click ok.



The image shows the Jenkins 'New Item' dialog. At the top, there's a search bar and a 'New Item' button. Below that, the 'Enter an item name' field contains 'caesar-cipher-pipelines'. Under 'Select an item type', several options are listed: Pipeline, Freestyle project, Multi-configuration project, Folder, and Multibranch Pipeline. The 'Multibranch Pipeline' option is highlighted with a blue border and a blue icon. An 'OK' button is at the bottom.

Jenkins / All / New Item

New Item

Enter an item name

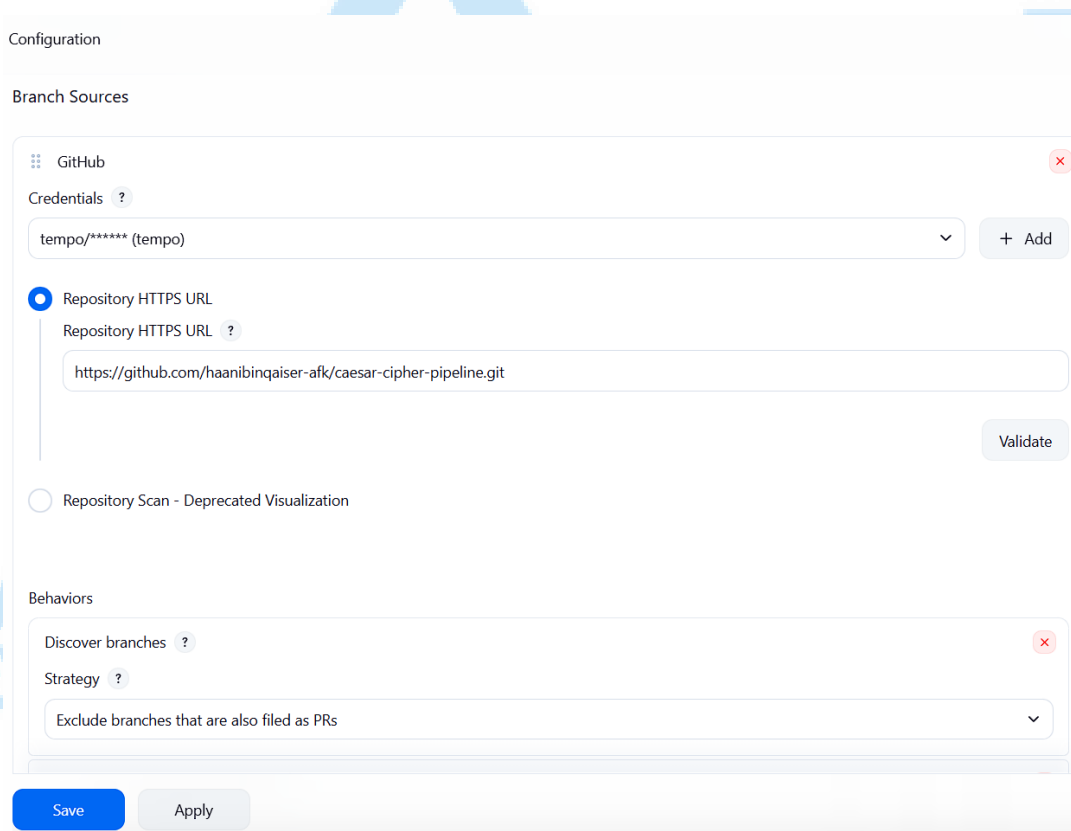
caesar-cipher-pipelines

Select an item type

- Pipeline**
Build, test, deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
- Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
- Folder
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.
- Multibranch Pipeline**
Creates a set of Pipeline projects according to detected branches in one SCM repository.

OK

Here scroll down to Branch sources and select Github as the source. Enter the Github repository link in the URL field.



The image shows the Jenkins 'Configuration' page for the 'caesar-cipher-pipelines' item. The 'Branch Sources' section is active, showing a 'GitHub' source. The 'Credentials' dropdown is set to 'tempo/***** (tempo)'. The 'Repository HTTPS URL' is set to 'https://github.com/haanibinqaiser-afk/caesar-cipher-pipeline.git'. The 'Behaviors' section shows 'Discover branches' set to 'Exclude branches that are also filed as PRs'. At the bottom, there are 'Save' and 'Apply' buttons.

Configuration

Branch Sources

GitHub

Credentials ?

tempo/***** (tempo) + Add

☒ Repository HTTPS URL

Repository HTTPS URL ?

https://github.com/haanibinqaiser-afk/caesar-cipher-pipeline.git

Validate

☐ Repository Scan - Deprecated Visualization

Behaviors

Discover branches ?

Strategy ?

Exclude branches that are also filed as PRs

Save Apply

Here it will ask you for credentials, credentials are must in this case. Select the first option and enter username and ID whatever you want. In my case, I have selected "tempo".

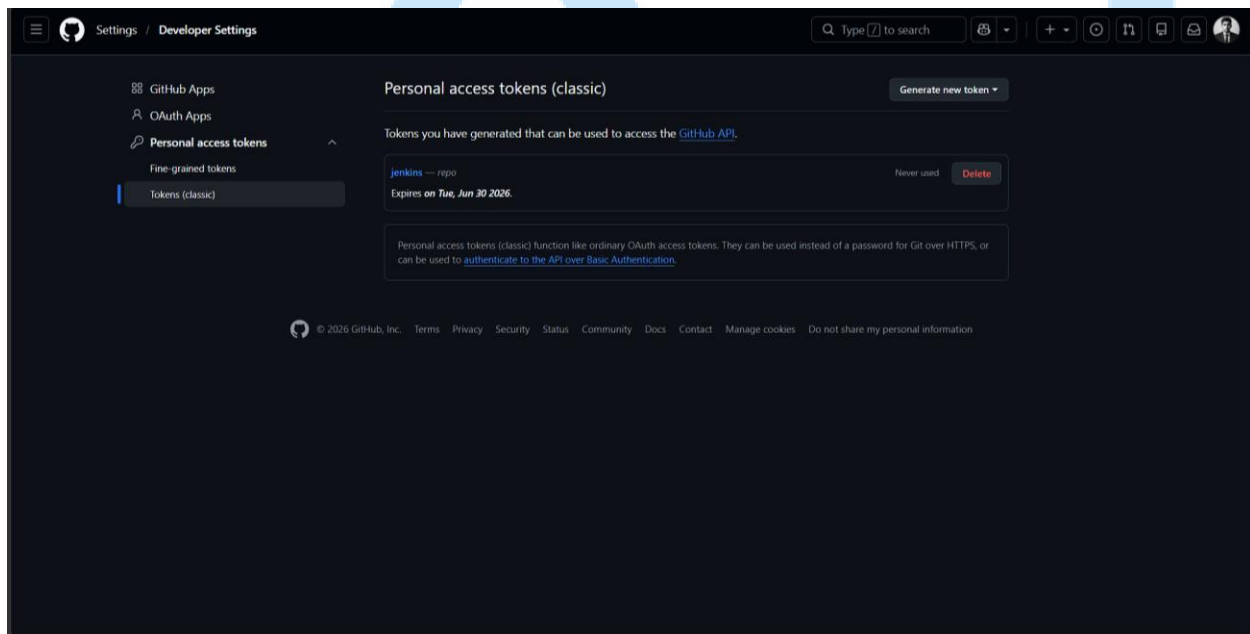
Add Credentials ✕

Select a type of credential

-  Username with password
Commonly used for authentication to services like Git, APIs, or registries.
-  GitHub App
-  SSH Username with private key
-  Secret file
-  Secret text
-  X.509 Client Certificate
-  Certificate
Upload a PKCS#12 or PEM encoded certificate and private key.

Next

But for password, we need to generate a token from Github. For that, go to Github developer Settings and generate a classic token, use it for the Jenkins Github source password as Github will only allow access through this token.



The screenshot shows the GitHub Developer Settings page for 'Personal access tokens (classic)'. The left sidebar contains a menu with 'GitHub Apps', 'OAuth Apps', 'Personal access tokens' (selected), 'Fine-grained tokens', and 'Tokens (classic)'. The main content area is titled 'Personal access tokens (classic)' and includes a 'Generate new token' button. Below this, it states 'Tokens you have generated that can be used to access the [GitHub API](#).' A table lists one token named 'jenkins' with a repository 'repo', 'Never used' status, and an expiration date of 'Expires on Tue, Jun 30 2026'. A 'Delete' button is next to the token name. A note at the bottom explains that classic tokens function like ordinary OAuth access tokens and can be used instead of a password for Git over HTTPS or to authenticate to the API over Basic Authentication.

Then go back to Jenkins configurations and scroll down to Build Configuration section, there, select the mode “by Jenkinsfile” and enter the script path as “Jenkinsfile”. Then save the configuration.

Build Configuration

Mode

by Jenkinsfile

Script Path ?

Jenkinsfile

Now the project is successfully implemented.

The screenshot shows the Jenkins web interface for a project named 'caesar-cipher-pipeline'. On the left is a sidebar with navigation links: Status (selected), Configure, Scan Repository Now, Scan Repository Log, Multibranch Pipeline Events, Delete Multibranch Pipeline, Build History, Project Relationship, Check File Fingerprint, GitHub, Rename, Config Files, Pipeline Syntax, and Credentials. Below these are sections for 'Build Queue' (empty) and 'Build Executor Status' (0/2). The main content area shows the project name and tabs for 'Branches (1)' and 'Pull Requests (0)'. A table displays build history for the 'main' branch:

S	W	Name	Last Success	Last Failure	Last Duration
✓	☁	main	13 hr #2	13 hr #1	1 min 58 sec

Below the table is a horizontal timeline and a legend for 'Icon: S M L'. The 'S' icon is highlighted. A green play button is visible on the right side of the table.

First click on the “Scan Repository Now” button to scan the repo.

AIR UNIVERSITY

The screenshot shows the Jenkins interface for the 'caesar-cipher-pipeline'. The left sidebar contains a menu with options: Status, Configure, Scan Repository Now, Scan Repository Log (selected), View as plain text, Multibranch Pipeline Events, Delete Multibranch Pipeline, Build History, Project Relationship, Check File Fingerprint, GitHub, Rename, Config Files, Pipeline Syntax, and Credentials. Below the menu is a 'Build Queue' section. The main content area is titled 'Scan Repository Log' with a green checkmark icon. It displays a log of the repository scan process, including the user 'temp', the time 'Sun May 31 21:09:09 PKT 2026', and the repository 'haanibinqaiser-afk/caesar-cipher-pipeline'. The log shows the process of getting remote pull requests, checking branches, and detecting changes on the 'main' branch. It concludes with 'Finished: SUCCESS'.

```
Started by user temp
[Sun May 31 21:09:09 PKT 2026] Starting branch indexing...
21:09:11 Connecting to https://api.github.com using tempo/***** (tempo)
Examining haanibinqaiser-afk/caesar-cipher-pipeline

Getting remote pull requests...

Checking branches...

Getting remote branches...

Checking branch main
'Jenkinsfile' found
Met criteria
Changes detected: main (6ad81ee414dc0b4a691f89da1ceb57ddfe57e28e → 7e6260cd6c90035457a64b239fad3f0a5f9fc4d2)
Scheduled build for branch: main

1 branches were processed

checking pull-requests...

0 pull requests were processed

Finished examining haanibinqaiser-afk/caesar-cipher-pipeline

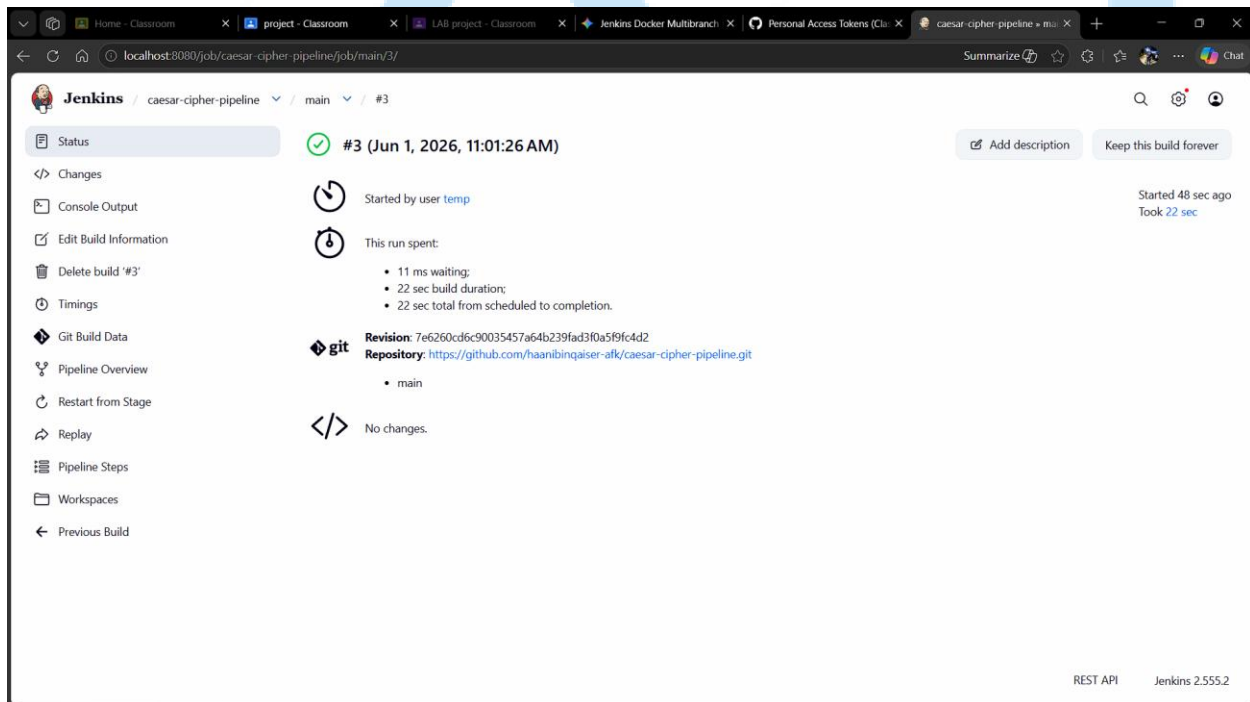
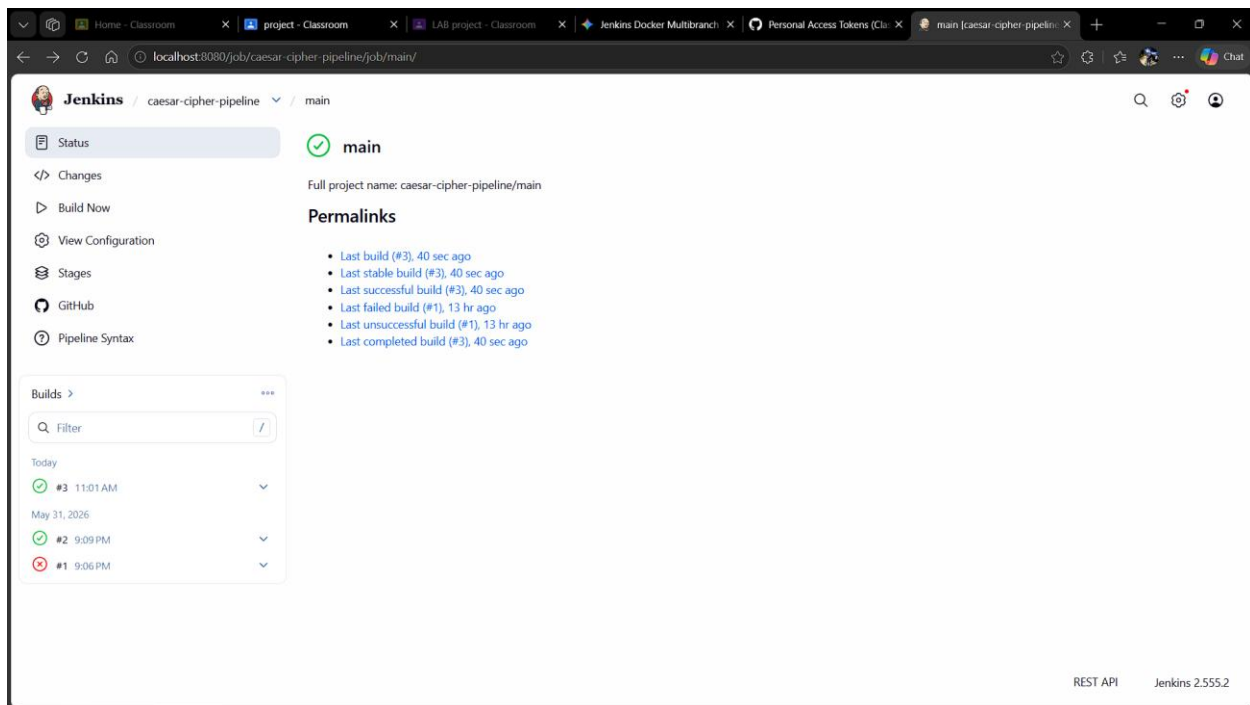
[Sun May 31 21:09:19 PKT 2026] Finished branch indexing. Indexing took 9.6 sec
Finished: SUCCESS
```

Then go to the Status section, there you will find the main branch. Click on it.

The screenshot shows the Jenkins 'Status' section for the 'caesar-cipher-pipeline'. The left sidebar is the same as in the previous screenshot. The main content area is titled 'caesar-cipher-pipeline' and shows a table of branches. The 'main' branch is listed with a green checkmark icon, indicating it is the current branch. The table also shows the last success and failure times, and the last duration. Below the table is a 'Build Queue' section and a 'Build Executor Status' section showing 0/2 executors.

S	W	Name	Last Success	Last Failure	Last Duration
✓	☁	main	13 hr #2	13 hr #1	1 min 58 sec

Then click the “Build Now” button. And then access the successful build from the Builds pane.



Then go to Console Output and then you can see the output of the Jenkins Automation system complete and fully functional.

Output:

Started by user `temp`

11:01:29 Connecting to <https://api.github.com> using `tempo/*****` (tempo)

Obtained Jenkinsfile from `7e6260cd6c90035457a64b239fad3f0a5f9fc4d2`

```
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\ProgramData\Jenkins\.jenkins\workspace\caesar-cipher-pipeline_main
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
The recommended git tool is: NONE
using credential tempo
> git.exe rev-parse --resolve-git-dir C:\ProgramData\Jenkins\.jenkins\workspace\caesar-cipher-
pipeline_main\.git # timeout=10
Fetching changes from the remote Git repository
> git.exe config remote.origin.url https://github.com/haanibinqaiser-afk/caesar-cipher-pipeline.git #
timeout=10
Fetching without tags
Fetching upstream changes from https://github.com/haanibinqaiser-afk/caesar-cipher-pipeline.git
> git.exe --version # timeout=10
> git --version # 'git version 2.54.0.windows.1'
using GIT_ASKPASS to set credentials tempo
> git.exe fetch --no-tags --force --progress -- https://github.com/haanibinqaiser-afk/caesar-cipher-
pipeline.git +refs/heads/main:refs/remotes/origin/main # timeout=10
Checking out Revision 7e6260cd6c90035457a64b239fad3f0a5f9fc4d2 (main)
> git.exe config core.sparsecheckout # timeout=10
> git.exe checkout -f 7e6260cd6c90035457a64b239fad3f0a5f9fc4d2 # timeout=10
Commit message: "Fix case-sensitivity typo in Dockerfile name"
> git.exe rev-list --no-walk 7e6260cd6c90035457a64b239fad3f0a5f9fc4d2 # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Verify Docker Accessibility)
[Pipeline] echo
Current Branch: main
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\caesar-cipher-pipeline_main>docker --version
Docker version 29.5.2, build 79eb04c
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Build Cipher Image)
[Pipeline] echo
Building the Caesar Cipher Docker executor...
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\caesar-cipher-pipeline_main>docker build -t caesar-
cipher:latest .
#0 building with "default" instance using docker driver

#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 126B 0.0s done
#1 DONE 0.0s
```

```
#2 [internal] load metadata for docker.io/library/python:3.11-slim
#2 DONE 4.6s

#3 [internal] load .dockerignore
#3 transferring context: 2B done
#3 DONE 0.0s

#4 [internal] load build context
#4 transferring context: 31B done
#4 DONE 0.1s

#5 [1/3] FROM docker.io/library/python:3.11-
slim@sha256:a3ab0b966bc4e91546a033e22093cb840908979487a9fc0e6e38295747e49ac0
#5 resolve docker.io/library/python:3.11-
slim@sha256:a3ab0b966bc4e91546a033e22093cb840908979487a9fc0e6e38295747e49ac0
#5 resolve docker.io/library/python:3.11-
slim@sha256:a3ab0b966bc4e91546a033e22093cb840908979487a9fc0e6e38295747e49ac0 0.1s done
#5 DONE 0.1s

#6 [2/3] WORKDIR /app
#6 CACHED

#7 [3/3] COPY cipher.py .
#7 CACHED

#8 exporting to image
#8 exporting layers done
#8 exporting manifest sha256:8000a22defdbf361d149cef2d98c4f20c0b81a61dd202a1be5d276f4ee3e6392
0.0s done
#8 exporting config sha256:61fa239784fb5b787cf40929bac4b217d5240999add8a420e24337a1c8e31353
0.0s done
#8 exporting attestation manifest
sha256:cac2f012a9269dab4e106e3ee1420cf4db64202515e8b8b86b9ea461f595a7ca 0.0s done
#8 exporting manifest list
sha256:f335d38ffed93d8ac3114c36d839dfc71e908a0c12511b67ab8f97ce48b9e457 0.0s done
#8 naming to docker.io/library/caesar-cipher:latest done
#8 unpacking to docker.io/library/caesar-cipher:latest 0.1s done
#8 DONE 0.3s
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Encryption Stage)
[Pipeline] echo
Original Text: DevOps with Jenkins, Docker Desktop, and WSL!
[Pipeline] echo
Encrypting...
[Pipeline] script
[Pipeline] {
[Pipeline] bat
[Pipeline] }
[Pipeline] // script
[Pipeline] echo
Encrypted Output: KlCVwz dpao Qlurpuz, Kvjrly Klzravw, huk DZS!
[Pipeline] }
```

```
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Decryption Stage)
[Pipeline] echo
Passing Encrypted Text to Decryption Stage...
[Pipeline] script
[Pipeline] {
[Pipeline] bat
[Pipeline] echo
Decrypted Output: DevOps with Jenkins, Docker Desktop, and WSL!
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Cleaning up local Docker images...
[Pipeline] bat
```

```
C:\ProgramData\Jenkins\jenkins\workspace\caesar-cipher-pipeline_main>docker rmi caesar-
cipher:latest --force
Untagged: caesar-cipher:latest
Deleted: sha256:f335d38ffed93d8ac3114c36d839dfc71e908a0c12511b67ab8f97ce48b9e457
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
```

GitHub has been notified of this commit's build result

Finished: SUCCESS

Project Overview:

The project uses a fully automated, containerized Continuous Integration pipeline (CI) that executes a classic Caesar cipher algorithm. This project illustrates Infrastructure as Code (IaC) and Pipeline as code ensuring that cryptographic operations are run isolated in a supportive environment without depending on the host machine's local runtimes.

Key Features:

Pipeline as Code: Managed entirely by the Jenkinsfile stored in source.

Containerized Execution: Uses docker to isolate the python cipher algorithm.

Secure SCM Integration: Configured as a multibranch pipeline connected securely to a private Github repository.

Ephemeral Environment Cleanup: Automatically removes temporary Docker image artifacts after execution to keep the environment clean and resource-efficient.

